

An FPGA-based Accelerator for 3D CNNs Using FFT convolution

Abstract—The high arithmetic complexity of 2D and 3D CNNs has led to new research regarding efficient hardware-based accelerators. This complexity stems from the convolutional layers of these networks which often rely on poorly-scaling algorithms, particularly in 3D. Winograd-based designs can achieve high resource efficiency in networks using small kernels, yet are limited by increasingly complex transform operations in higher dimensions. FFT convolution has seen recent success in applying decomposition algorithms, such as overlap-add, to CNNs, increasing its efficiency for large inputs. Likewise, the Complex-domain multiplications required by the algorithm can be optimized using polar coordinate representation. Our analysis reveals the lower asymptotic bound of overlap-add FFT compared to Winograd convolution. We propose a 3D CNN accelerator using FFT convolution augmented by overlap-add and CORDIC algorithms. We implement and test this design on FPGA technology using RTL synthesis, achieving over twice the DSP efficiency of other FPGA-based state-of-the-art accelerators.



1 INTRODUCTION

IMAGE processing demands have increased substantially in the last decade, due in part to the success of Convolutional Neural Networks (CNNs) in two-dimensional image recognition. Several accelerator architectures have been developed to attend to these demands; a method that has seen wide success in 2D [1] and 3D [2] CNNs.

A variety of optimization approaches have been developed for the task of accelerating CNNs, including: efficient memory access [3], [4], [5]; parallelization methods [6], [7] often via systolic-array inference [8]; optimizing floating-point operations [9]; quantizing weights and activations [10], [11]; and addressing data sparsity [4], [9].

One approach that has also garnered attention is the application of more efficient convolution algorithms. The standard discrete convolution algorithm, as well as the more widely used matrix-multiplication (MMX), have a base asymptotic complexity of $O(N^2)$, scaling very poorly with image and kernel sizes. This is particularly relevant to 3D CNNs, as the processing load of its convolution layers can quickly surpass all other layers in the network [12].

In 2016, Lavin & Gray [13] reviewed the relevant convolution algorithms—which were, at the time, MMX and FFT convolution—and proposed a new class of algorithms based on the Coppersmith–Winograd series of matrix multiplication methods [14]. Similar algorithms with lower asymptotic bounds have been discovered [15], [16], however, they incur increasingly complex transforms which severely hinders their applicability. As such, the so-called Winograd class of convolution algorithms has become the standard for hardware-efficient CNN designs [17], [18], [19]. Lavin & Gray deemed FFT convolution inefficient, as its upsides only overweight its downsides for convolutions using large kernels [13], a method which has lost prevalence to deeper networks using small (usually 3×3) kernels [20]. The algorithm is also not easily implemented in high-level designs, due in large part to its reliance on Complex-domain operations.

Since then, new techniques have been found to optimize key parts of the algorithm, such as convolution splitting [21], overlap-add [22], [23], mixed-radix FFT [24], and spectral

CNNs [25], [26], [27], [28], [29]. It has also found success in CNNs required the use of large kernels, such as 3D CNNs [30], [31], [32].

There remains, however, the issue of Complex-domain multiplication. Recent FPGA implementations have addressed this issue using parallelization [33] and pipelining [9]. These methods, however, do not directly address the high operation count of Complex multiplication, and instead deploy extra hardware resources to overcome it. As such, their hardware efficiency (measured in throughput per component) remains the same.

It is possible to reduce the operation count of Complex multiplication by representing its inputs as polar coordinates [34]. This was first implemented for CNNs in 2024 by Reis *et al.* [35] using high-level synthesis (HLS) and achieving promising results. However, HLS, while an important tool for quickly implementing designs, has been frequently criticized in CNN applications regarding resource underutilization [36], [37], [38], high power consumption [39] and general performance issues [40], [41], [42]. This is opposed to RTL models, written in hardware-description languages (HDL), where low-level optimizations are available at the cost of longer development cycles.

We propose an FFT-based 3D-convolution accelerator using hardware-efficient polar-coordinate multiplication, an FFT pipeline, and optimized for large images using input slicing and overlap-add. We synthesize the architecture using HDL and implement it in a VC709 FPGA board. By validating our implementation using state-of-the-art benchmarks, we demonstrate the viability of FFT convolution in non-spectral 3D CNNs.

This document is organized as follows: in Section 2 we discuss the state-of-the-art convolution algorithms and their computational complexity; in Section 3 we describe the design choices, optimizations, and testing procedures of our proposed architecture; Section 4 lists the results obtained and comparisons to the state-of-the-art; Section 5 discusses related work; and Section 6 presents our conclusions.

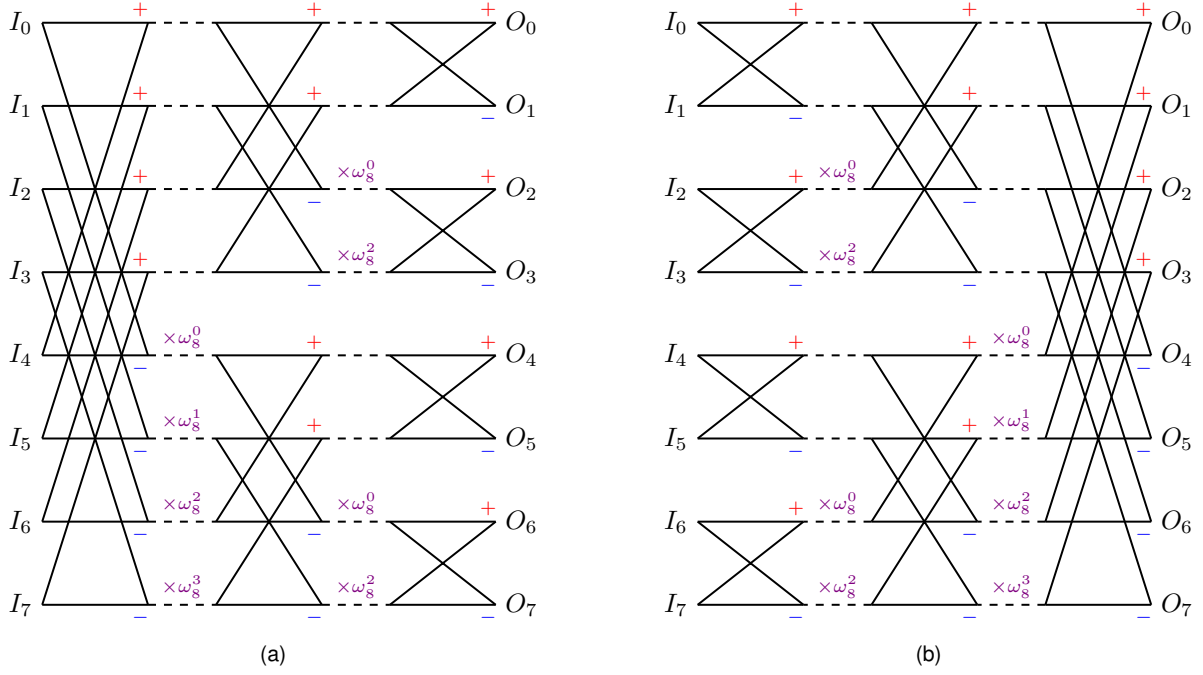


Fig. 1. Visual representation of an 8-point radix-2 FFT using decimation in frequency (a) and decimation in time (b).

2 CONVOLUTION ALGORITHMS

Lavin & Gray [13], as well as later publications [17], have performed comparative algorithmic analyses of MMX, Winograd, and FFT convolution. We will add to their analyses by addressing the scalability of these algorithms and how this property affects the total complexity of the system.

Let us first define a standard nomenclature:

- I is a 3D input channel with dimensions $\{I_x, I_y, I_z\}$. There are I_c input channels. Each of its elements are defined $I[ix, iy, iz, c]$.
- K is a 3D convolution kernel with dimensions $\{K_x, K_y, K_z\}$. There are K_f kernels. Each of its elements are defined $K[kx, ky, kz, f]$.
- $I[c] * K[f]$ is the convolution of one channel I and one kernel K . There are I_c output channels and K_c filter maps.

The output dimensions are a function of the respective sizes of I and K and depend on the *mode* of convolution, as follows:

- *Full*: $O_d = I_d + K_d - 1$
- *Same*: $O_d = I_d$
- *Valid*: $O_d = I_d - K_d$

The baseline method of convolution used by CNNs, discrete convolution, uses *Same* mode. It is defined as

$$I[c] * K[f] = \sum_{z=0}^{K_z} \sum_{y=0}^{K_y} \sum_{x=0}^{K_x} I[ox + kx, oy + ky, oz + kz, c] K[x, y, z, f]. \quad (1)$$

This method shares its complexity with the more commonly used MMX algorithm [34]. We will calculate the total

number of multiplications required by each algorithm, labeled M . For discrete and MMX convolutions, this number is equal to the product of each dimension processed, i.e.,

$$M = I_x I_y I_z I_c K_x K_y K_z K_f. \quad (2)$$

2.1 FFT convolution

Lower complexity can be achieved via the Convolution Theorem:

$$I[c] * K[f] = \mathcal{F}^{-1}(\mathcal{F}(I[c]) \cdot \mathcal{F}(K[f])), \quad (3)$$

where $\cdot$ represents the pointwise product (also known as *Hadamard* product), and $\mathcal{F}$ and $\mathcal{F}^{-1}$ represent the Discrete Fourier Transform (DFT) and its inverse (IDFT). This method outputs *full* mode convolution.

The class of algorithms known as *Fast Fourier Transforms* (FFT) are used to compute the DFTs. By definition, an FFT computes an N -point DFT in $O(N \log_2 N)$. The standard radix-2 FFT [43], [44] breaks down the DFT into $\log_2 N$ steps, each consisting of multiple “butterfly operations” paired with twiddle-factor multiplications. The order of operations depends on the recursion method used: Decimation in Frequency (DIF) or Decimation in Time (DIT), as illustrated by Figures 1a and 1b, respectively. Each “butterfly” receives a pair (A, B) and outputs $(A + B, A - B)$, where A and B are Complex numbers. Each twiddle factor multiplier calculates a product by a root-of-unity ω with the form

$$\omega_N^k = e^{\frac{2\pi i}{N} k}, \quad (4)$$

where $k \in \mathbb{Z} \cap [0, N - 1]$.

While the input signal of the FFT is one-dimensional, it is possible to represent an arbitrary number of dimensions using zero-padding. Figure 2a represents a signal K given its dimensions all have size 2. To convolve this array with

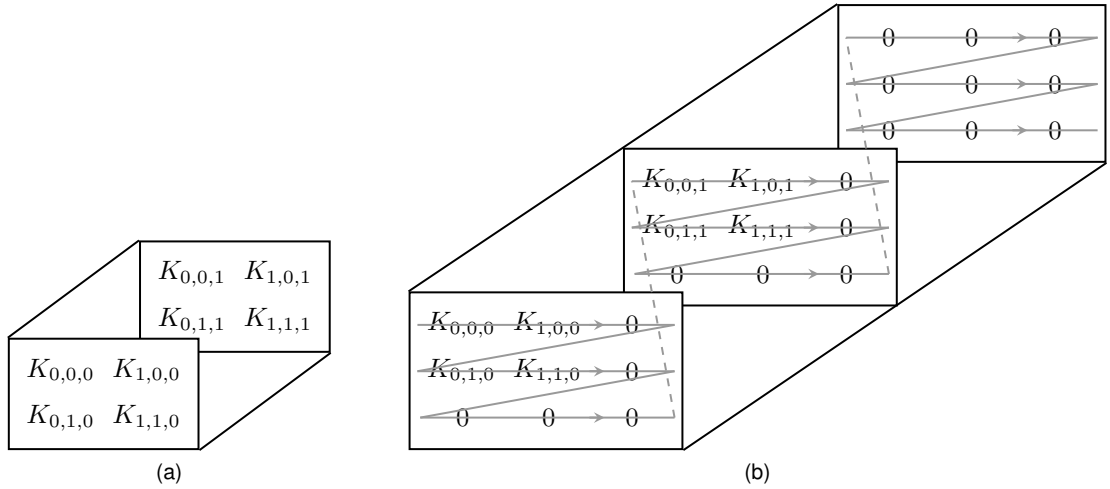


Fig. 2. A three-dimensional signal (a) and the required zero-padding to avoid cyclic convolution (b).

another signal of the same shape, we must first pad each dimension d by $2d - 1$, thus conveying its dimensionality and avoiding cyclic convolution. Figure 2b represents the padded signal; the gray line displays the order of rasterization.

The rasterized signal is further padded to its next power of two, so that radix-2 FFT can be applied across $\log_2 N$ stages. The size of the resulting array, denoted N , as a function of the sizes of K , is

$$N = 2^{\lceil \log_2(2K_x - 1) \times \log_2(2K_y - 1) \times \log_2(2K_z - 1) \rceil}, \quad (5)$$

where $\lceil \cdot \rceil$ indicates the *ceiling* function.

The transformed array exiting the FFT chain is subject to *hermitian symmetry*. Assume an N -point array of Complex numbers of the form $(a + bi)$. If the array has hermitian symmetry, the points with indexes $[N/2 + 1]$ to $[N - 1]$ are equivalent to the Complex conjugate of the points with indexes $[N/2 - 1]$ to 1, respectively. This means that almost half of the array is redundant, and can be inferred by switching the sign bit of its y-component. This property is preserved after the pointwise multiplication, and as such the total number of pointwise Complex multiplications required by FFT convolution is

$$M(\mathbb{C}) = \frac{N}{2} + 1. \quad (6)$$

In order to convolve I and K in this manner, the sizes of each transformed array must be equal. This can be achieved by zero-padding the smaller signal to match the larger one, however, this is inefficient in CNN applications where I is much larger than K . Instead, we can “slice” the input into kernel-sized pieces and convolve each slice with the kernel. The overlapping points of each convolution are added together, forming the complete output. This is known as *overlap-add* [23], [45].

Let S be a 3D slice of I with dimensions $\{K_x, K_y, K_z\}$. The number of slices per dimension is $\{S_x, S_y, S_z\}$, such that

$$S_x = \left\lceil \frac{I_x}{K_x} \right\rceil; \quad S_y = \left\lceil \frac{I_y}{K_y} \right\rceil; \quad S_z = \left\lceil \frac{I_z}{K_z} \right\rceil. \quad (7)$$

Complex-domain multiplication can be performed with 4 Real-domain multiplications. Let X and Y be Complex numbers such that $X = a + bi$ and $Y = c + di$. The product of the two signals is:

$$XY = (a + bi)(c + di) = (ac - bd) + (ad + bc)i. \quad (8)$$

Using this method, the total number of multiplications required by FFT convolution is

$$M = 4 \left\lceil \frac{I_x}{K_x} \right\rceil \left\lceil \frac{I_y}{K_y} \right\rceil \left\lceil \frac{I_z}{K_z} \right\rceil \left(\frac{N}{2} + 1 \right). \quad (9)$$

However, X and Y can also be represented in polar coordinates such that $X = A/\alpha_1$ and $Y = B/\alpha_2$, where A, B are radial distances and α_1, α_2 are angular coordinates. As such

$$XY = A/\alpha_1 \cdot B/\alpha_2 = AB/\alpha_1 + \alpha_2. \quad (10)$$

This representation reduces the total operation count to 1 Real multiplication and 1 Real addition. Section 3 describes the algorithm used to implement the conversions between these coordinate systems. Notably, these conversions do not incur any multiplications, and as such

$$M = \left\lceil \frac{I_x}{K_x} \right\rceil \left\lceil \frac{I_y}{K_y} \right\rceil \left\lceil \frac{I_z}{K_z} \right\rceil \left(\frac{N}{2} + 1 \right). \quad (11)$$

2.2 Winograd

Similarly to FFT convolution, the Winograd method transforms the input and kernel into a domain (more strictly, a *field*) where pointwise multiplication is equivalent to the convolution of the original signals [13]. In three dimensions, the algorithm defines convolution as $F(m^3, r^3)$, where m and r are integers. Let A and B be matrices that contain $I[c]$ and $K[f]$ such that their total size is equal to $(m + r - 1)^3$ and r^3 , respectively. Convolution is then expressed as

$$A * B = (Z((XAX^T)^R X^T \cdot (YAY^T)^R Y^T)Z^T)^R Z^T,$$

where X, Y , and Z are transform matrices, the superscript R denotes clockwise rotation, and T denotes transposing. The result of this operation has size m^3 and outputs a *valid* mode convolution.

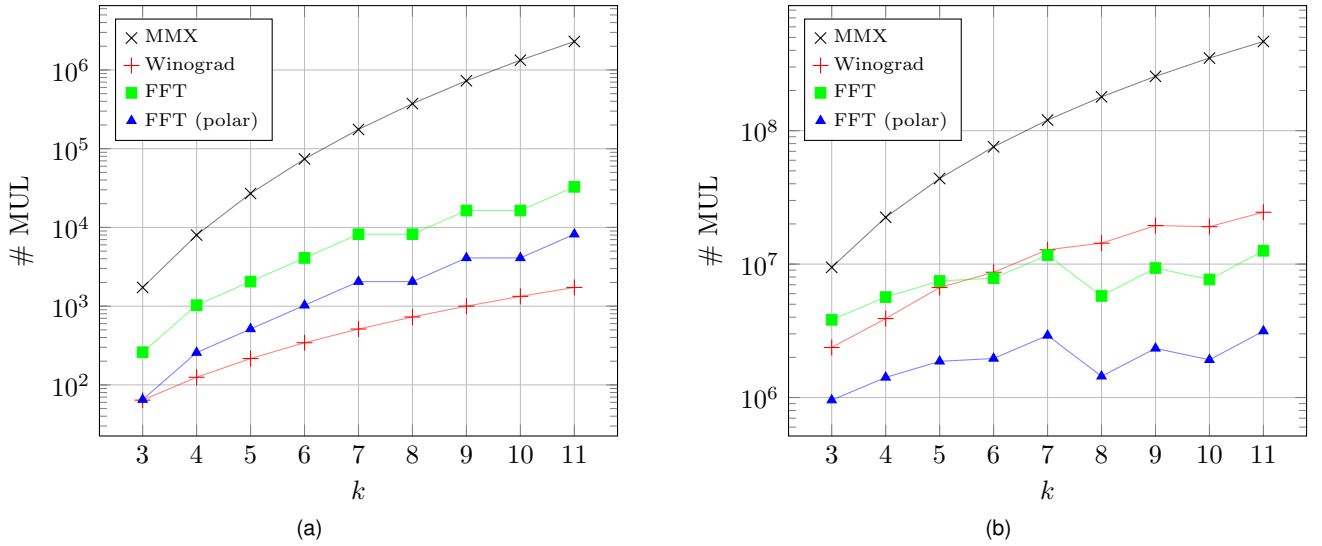


Fig. 3. Number of multiplications (log scale) vs kernel-size for $I = 4 \times 4 \times 4$ (a) and $I = 128 \times 171 \times 16$ (b).

The transform matrices depend on m , r , and the point-selection procedure used. 3D CNN applications require at least $F(2^3, 3^3)$ and trivial matrices for this operation have been found, such as the set used by Shen *et al.* (2018) [17]:

$$X = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}, \quad Y = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix},$$

$$Z = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}.$$

These matrices are considered *trivial* because their multiplication by another matrix is computationally efficient: multiplying by 0 or 1 is trivial and multiplying by $\frac{1}{2}$ can be computed as a right-bit-shift. Higher values of m and r incur different matrices where this property cannot be guaranteed.

It is possible to calculate multiple $F(m^3, r^3)$ convolutions and add the overlapping points to form a complete output [31]. However, this approach differs from the FFT method in that it requires calculating one convolution per stride of the kernel, as the output is *valid* type. For an image I and stride 1, the total number of multiplications is

$$\left\lceil \frac{I_x - r}{m} \right\rceil \left\lceil \frac{I_y - r}{m} \right\rceil \left\lceil \frac{I_z - r}{m} \right\rceil (m + r - 1)^3. \quad (12)$$

2.3 Comparison

Figures 3a and 3b are plots of Equations 2 (baseline), 12 (Winograd), 9 (FFT convolution), and 11 (FFT convolution using polar multiplication), for different kernel sizes and assuming K is equilateral and has size k . Figure 3a assumes $I = (k + 1)^3$, or one semi-convolution. Figure 3b assumes $I = 128 \times 171 \times 16$, the input size used by C3D [46], a 3D CNN widely used as a benchmark.

These graphs do not account for the multiplications stemming from Winograd's transform at higher kernel-sizes, nor the twiddle-factor multiplications used by FFT convolution, as both of them are multiplications by constants and are optimized at synthesis time.

3 IMPLEMENTATION

In order to convert the image and kernel between rectangular and polar coordinates, we implement CORDIC (COordinate Rotation DIgital Computer) [47], a trigonometric algorithm that iteratively calculates the rotation of a given vector by decreasingly-sized angles. It has two modes of operation: rotation and vectorization; as well as three inputs, here labeled (x, y, z) . In vectorization mode, the rectangular coordinates of a Complex number are inputted as (x_0, y_0) , and after j iterations, the radial distance and angular coordinate are outputted as (x_j, z_j) , respectively. Similarly, in rotation mode, the polar coordinates of a Complex number are inputted at (x_0, z_0) , and after j iterations, the respective rectangular coordinates are outputted as (x_j, y_j) . This is generally expressed as

$$\begin{bmatrix} x_{j+1} \\ y_{j+1} \\ z_{j+1} \end{bmatrix} = \begin{bmatrix} x_j - \sigma_j 2^{-j} y_j \\ y_j + \sigma_j 2^{-j} x_j \\ z_j + \sigma_j \arctan(2^{-j}) \end{bmatrix}, \quad (13)$$

where

$$\sigma_j = \begin{cases} \text{sign}(y_j), & \text{in vectorization mode} \\ -\text{sign}(z_j), & \text{in rotation mode} \end{cases}. \quad (14)$$

The precision of the algorithm is equivalent to 2^{-j} , or one bit of precision per iteration. Note that when using fixed-point notation, it is sufficient to iterate once for each bit after the point. We denote the total number of bits of a variable as S_t and the number of bits after the point as S_p . Our implementation defaults to 24 and 16, respectively.

A simple hardware implementation of CORDIC consists of two S_t -bit adders, one S_p -bit adder, two variable

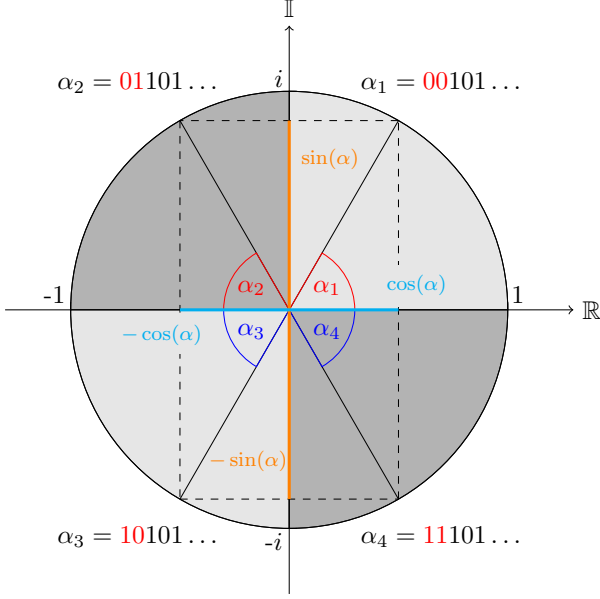


Fig. 4. Four symmetric angles encoded as π -fractions.

bit-shifters, three registers, and a lookup table for storing $\arctan(2^{-j})$.

The outputs of x_j and y_j are scaled by a constant κ_j on each iteration, where

$$\kappa_j = \cos(\arctan(2^{-j})), \quad (15)$$

and must be corrected to obtain the desired result. The correction can be made after the process completes, by multiplying each output by the scaling factor

$$\frac{1}{\kappa} = \prod_{j=0}^{S_p-1} \frac{1}{\sqrt{1+2^{-2j}}} \stackrel{S_p=16}{\approx} 0.60725. \quad (16)$$

In the pointwise multiplication step, CORDIC is used 3 times. The first two convert the image slice and kernel to polar coordinates and the last converts the result back to rectangular coordinates. The scaling factor carries over across operations, including pointwise multiplication, and as such we can apply the correction at any point in the process (image, kernel, or product). The combined scaling factor is

$$\frac{1}{\kappa^3} \stackrel{S_p=16}{\approx} 0.22393. \quad (17)$$

The convolution requires adding two angles before the transformation back into rectangular coordinates, which can cause issues if the resulting angle exceeds π radians. To circumvent this, we store the *quadrant* of each angular coordinate in the unit circle, normalize them to the first quadrant, perform the add operation, and convert the result into its appropriate quadrant. To simplify this computation, we divide each $\arctan(2^{-j})$ by 2π . The resulting binary, encoded in 16-bits, represents fractions of π , with its two most-significant bits representing the quadrant of the vector in the unit circle, as shown in Figure 4. This allows quick access to the quadrant of the angle, as well as enabling normalization to any quadrant by simply setting its first two bits.

CORDIC's complexity does not increase with the image- or kernel-sizes, depending instead on S_p which gives it a relative asymptotic complexity of $O(1)$ and enables a trade-off between numerical precision and arithmetic complexity.

CORDIC can also be applied to the FFTs to more efficiently calculate twiddle-factor multiplications [48], [49]. Each CORDIC iteration rotates the input by elementary angles which coincide with the twiddle factors used by the FFT. This, however, scales each point that wasn't multiplied by a twiddle factor by κ . Correcting each point results in an increase of LUT components used. Considering this trade-off, we opted to implement the twiddle-factor rotations using DSP blocks, in order to balance the DSP and LUT costs.

Our twiddle-factor multiplication design using DSPs is a variant of the Karatsuba multiplication algorithm [50]. Using the previously established nomenclature for Complex numbers, Karatsuba defines three intermediate variables

$$\begin{cases} k_1 = c \times (a + b) \\ k_2 = b \times (c + d) \\ k_3 = a \times (d - c) \end{cases}, \quad (18)$$

such that

$$XY = (k_1 - k_2) + (k_1 + k_3)i. \quad (19)$$

This algorithm incurs 5 additions and 3 multiplications, however, in the case of twiddle-factor multiplication, c and d are constant. By extension, $(c + d)$ and $(d - c)$ are also constant, and can be pre-calculated and stored in a lookup table. This lowers the operation count to 3 adds and 3 muls, a substantial improvement over the standard 2 adds and 4 muls of Equation 8.

3.1 Pipelining

Each step of the proposed design can be pipelined to greatly increase throughput. Figure 5 illustrates our pipelined design, optimized for hardware efficiency. The system consists of two phases, as delimited by the green line. Hardware across phases is reused instead of replicated.

In Phase-I, the system rasterizes and pads the 3D slice as shown in Figure 2, then multiplexes it into the FFT chain.

The FFT is divided into $\log_2 N$ stages, where N is defined by Equation 5. Each stage consists of one butterfly, one twiddle-factor multiplication block, and a set of registers. The data entering the first FFT stage is still on the Real domain. As such, we arranged this FFT in decimation in frequency (see figure 1a), as this mode places most non-redundant twiddle-factor multiplications at the first stage, significantly simplifying these operations.

The add and subtract blocks within the butterflies are (on every layer except the first) Complex operation blocks, and as such constitute two operations each. We implemented the twiddle-factor multiplication blocks (past the first stage) using Karatsuba, resulting in 3 adders and 3 multipliers each, along with a lookup table that is shared across stages. Many of these operations will be optimized away by the synthesizer, as about half the input array is zeroes. Some twiddle factors are also trivial to multiply, such as $\omega^0 = 1$ and $\omega^{\frac{N}{4}} = i$.

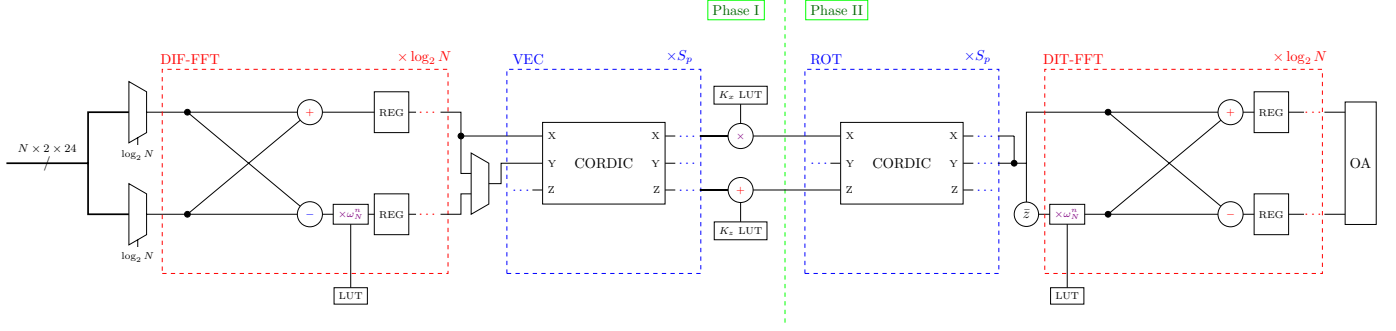


Fig. 5. Fully-pipelined FFT convolution design using overlap-add and CORDIC.

Each FFT node processes two input points per machine cycle and stores the result into the register bank. On the next cycle, the node processes two more points, the next stage reads the result, and the process repeats along the chain. Once the pipeline is full, the FFT chain is capable of processing two input points per cycle.

Hermitian symmetry carries over through CORDIC and polar multiplication. As such, the points past $N/2$ can be inferred much later. The edge case for index $N/2$ still needs to be calculated, as it is not the Complex conjugate of index 0. Its insertion into the next block is done via a multiplexer.

The CORDIC chain is set to vectorization on Phase-I to perform the conversion to polar coordinates. There are S_p CORDIC blocks which, when fully pipelined, process two points (one symmetrical) of the original array.

The polar multiplication step consists of one multiplier and one adder, in accordance to Equation 10. The kernels are processed at synthesis time and their Complex polar coordinates are stored in their respective lookup tables. Before being stored, each kernel's radial distance is multiplied by κ (see Equation 17), which applies the correction to each point of the array without the need for additional multiplier blocks.

In Phase-I, each slice S of each channel I_c is processed up to the multiplication by K and is stored in a register bank. After the process is complete, Phase-II begins, where the CORDIC chain is set to rotation mode, and the FFT is rearranged into DIT mode. This process is managed by a control unit and routing elements, not depicted in the diagram to avoid clutter.

The output of CORDIC during Phase-II is joined at the final stage, forming one point of the N -sized array. The respective Complex conjugate (denoted $\bar{z}$) is inferred and sent into the FFT chain. The $N/2$ exception to hermitian symmetry is always a Real number whose polar-coordinate conversion is trivial and need not be processed by CORDIC.

The FFT chain then executes the inverse transform. The last stage of DIF is equivalent to the first stage of DIT (see Figure 1), allowing data to flow seemingly along the pipeline. As the mirror to DIF, DIT places most non-redundant twiddle-factor multiplications at the end. The data leaving the system is on the Real domain, as all Complex parts will cancel-out, simplifying these operations as well.

Finally, the overlap-add module (OA) receives the two Real points and adds them to the respective indexes in the

TABLE 1
Resource usage of pipelined and parallel cores in VC709 environment.

Core	DSP	LUT	FF
Pipelined	1	5861	4265
[usage]	0.03%	1.3%	0.5%
Parallel	65	296572	213376
[usage]	1.8%	66.9%	25.6%

feature map. This process is repeated for all channels and all kernels. Consequently, Phase-II is K_f times longer than Phase-I, and if K_f is large, Phase-II is responsible nearly all processing time.

3.2 Parallelism

The design in Figure 5 is compact and scalable, useful for implementation across a range of FPGA devices. However, we can further improve the hardware efficiency of the design using parallelism.

Previously, we synthesized one butterfly and twiddle-factor multiplier per FFT stage. We can instead synthesize every block on each stage, allowing the synthesizer to better optimize the redundant twiddle-factor multiplications.

To maintain uninterrupted data flow along the pipeline, we scale the CORDIC chain accordingly. Similarly to the FFT chain, many CORDIC blocks will become redundant and will be optimized away during synthesis. We also parallelize the polar multiplication and overlap-add stages, which now complete one slice per cycle in Phase-II.

We label this version of the architecture **parallel-core**, while the design described in Section 3.1 is accordingly labeled **pipelined-core**.

3.3 Synthesis & Testing

We synthesized and implemented both designs on a VC709 FPGA board. The system achieved timing closure at 150 MHz for parallel-core and 200 MHz for pipelined-core, due to routing and congestion factors.

We carried out software testing to ascertain correctness and measure precision, done via a randomized trial and comparing its results to a standard convolution algorithm at each iteration. We also performed edge-case trials using very large and very small inputs.

TABLE 2
Comparison of different FPGA accelerator architectures applied on C3D.

Project	Shen <i>et al.</i> (2018) [17]		Liu <i>et al.</i> (2019) [51]	Fan <i>et al.</i> (2021) [9]	Yang <i>et al.</i> (2023) [52]	Our Design
Algorithm	Winograd		MMX	MMX	MMX	FFT
Precision	16-fixed		16-fixed	<16-float	(unmentioned)	16-fixed
Board	VC709	VUS440	VC709	GX1150	XCZU15	VC709
Technology (nm)	28	20	28	20	16	28
Frequency (MHz)	150	200	120	220	100	150
DSP	1536	1536	3595	1345	1317	65
Throughput (GOP/s)	474.3	940.6	667.7	1536	209.4	107.8
Resource efficiency (20)	0.31	0.61	0.19	1.14	0.16	1.66
Normalized (21)	2.06	3.06	1.55	5.19	1.59	11.05

Table 1 displays the resource usage metrics of both cores for $K = 3 \times 3 \times 3$.

Throughput (GOP/s) was measured using the first convolutional layer of C3D [46], where $I = 128 \times 171 \times 3$, $I_c = 3$, and $K_f = 16$. Resource efficiency was measured as the product of throughput and number of DSPs inferred [9], [37], [53], [54]. We express this as

$$\frac{GOP}{s \times DSP}. \quad (20)$$

This metric can be normalized across different FPGA devices by dividing it by the operational frequency [37], [55], [56]. We express this as

$$\frac{GOP}{s \times DSP \times MHz}. \quad (21)$$

4 RESULTS

Software testing using 24-bit fixed-point values with 16 fractional bits revealed a worst-case precision of 2^{-8} . In other words, the 16 most significant bits always returned the correct value across both the randomized and edge-case trials. This is sufficient for both training and inference of CNNs [57].

To the extent of our research regarding FPGA-based acceleration of 3D CNNs, Fan *et al.* (2021) [9] achieves the highest resource efficiency. Their choice of convolution algorithm is the standard matrix-multiplication, however, the implementation of floating-point quantization enables more efficient operations within the algorithm. Shen *et al.* (2018) [17] remains the most resource-efficient FPGA-based accelerator that uses Winograd convolution and that is capable of processing 3D convolutional filters.

Table 2 displays the throughput (GOP/s), hardware efficiency (20), and normalized hardware efficiency (21) of our parallel-core design and the state-of-the-art. This comparison is analogous to the ones made by previous projects in their analyses [9], [17]. Liu *et al.* [51] and Yang *et al.* [52] have also contributed to the state-of-the-art and were included in this comparison.

5 RELATED WORK

CNN acceleration is a broad and actively-researched field. This section covers some relevant bibliography not previously cited.

Software-based network architectures and hardware accelerators can be used in tandem to achieve higher performance and efficiency. Spectral CNNs [25], [26], [27], [28], [29] as well as rotation-invariant CNNs [58], [59] avoid the need to transform between the Real and Complex domains when using FFTs. This incurs changes across each layer of the network, and more research is needed to evaluate whether the impacts on accuracy and development cycles are worth the performance gain.

Since Shen *et al.* (2018) [17], algorithmic optimizations of the Winograd method have been published [18], [60], [61] but have not yet been implemented in an FPGA-based accelerator. There exist more recent applications of the algorithm for CNNs [37], [62], but they are not trivially expandable to three dimensions.

Winograd and FFT convolutions may be able to benefit from similar quantization methods as the ones used by Fan *et al.* [9]. Their implementation does not optimize the total number of operations of the algorithm, instead optimizing the operations themselves, leaving room for algorithmic optimizations.

The possibility of designing CNNs using CORDIC-based activation functions has been proposed [63], [64]. This would likely lead to more optimal accelerators using FFT convolution.

There exist other FFT algorithms, such as split-radix [65] and prime-factor [66] FFTs, which might be useful in optimizing slice dimensions.

6 CONCLUSIONS & FUTURE WORK

We demonstrate the lower complexity bounds of FFT convolution using overlap-add, in comparison to related algorithms. We design and implement an FPGA-based accelerator using polar-coordinate multiplication, achieving state-of-the-art resource efficiency.

Our design's significant LUT and FF usages stem from the twiddle-factor multiplications on the FFT pipeline. Further algorithmic optimizations may be possible, such as the ones listed in Section 5. Applications in more recent FPGA devices will also likely yield further resource optimization.

Based on the data presented in this work, we suggest that FFT convolution, paired with overlap-add and polar multiplication, is the method of choice for efficiently accelerating 3D CNNs and other networks reliant on large kernels.

REFERENCES

- [1] S. Mittal, "A survey of fpga-based accelerators for convolutional neural networks," *Neural computing and applications*, vol. 32, no. 4, pp. 1109–1139, 2020.
- [2] S. Mittal and Vibhu, "A survey of accelerator architectures for 3d convolution neural networks," *Journal of Systems Architecture*, vol. 115, p. 102041, 2021.
- [3] X. Han, D. Zhou, S. Wang, and S. Kimura, "Cnn-merp: An fpga-based memory-efficient reconfigurable processor for forward and backward propagation of convolutional neural networks," in *2016 IEEE 34th International Conference on Computer Design (ICCD)*. IEEE, 2016, pp. 320–327.
- [4] C. Jiang, D. Ojika, B. Patel, and H. Lam, "Optimized fpga-based deep learning accelerator for sparse cnn using high bandwidth memory," in *IEEE Annual International Symposium on Field-Programmable Custom Computing Machines*, 2021.
- [5] M. Doumet, M. Stan, M. Hall, and V. Betz, "H2pipe: high throughput cnn inference on fpgas with high-bandwidth memory," in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2024, pp. 69–77.
- [6] N. P. Jouppi, D. H. Yoon, G. Kurian, S. Li, N. Patil, J. Laudon, C. Young, and D. Patterson, "A domain-specific supercomputer for training deep neural networks," *Communications of the ACM*, vol. 63, no. 7, pp. 67–78, 2020.
- [7] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "Nvidia a100 tensor core gpu: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.
- [8] S. H. Chua, T. H. Teo, M. A. Tiruye, and I.-C. Wey, "Systolic array based convolutional neural network inference on fpga," in *2022 IEEE 15th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*. IEEE, 2022, pp. 128–133.
- [9] H. Fan, S. Liu, Z. Que, X. Niu, and W. Luk, "High-performance acceleration of 2-d and 3-d cnns on fpgas using static block floating point," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 4473–4487, 2021.
- [10] Y. He, W. Liu, M. Tahir, Z. Li, S. Zhang, and H. B. Amur, "Research on efficient cnn acceleration through mixed precision quantization: A comprehensive methodology," *International Journal of Advanced Computer Science & Applications*, vol. 14, no. 12, 2023.
- [11] O. B. H. Salah, S. Messaoud, M. A. Hajjaji, M. Atri, and N. Liouane, "Post-training quantization for efficient fpga-based neural network acceleration," *Integration*, p. 102508, 2025.
- [12] K. Hegde, R. Agrawal, Y. Yao, and C. W. Fletcher, "Morph: Flexible acceleration for 3d cnn-based video understanding," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 933–946.
- [13] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4013–4021.
- [14] D. Coppersmith and S. Winograd, "On the asymptotic complexity of matrix multiplication," *SIAM Journal on Computing*, vol. 11, no. 3, pp. 472–492, 1982.
- [15] E. Karstadt and O. Schwartz, "Matrix multiplication, a little faster," *Journal of the ACM (JACM)*, vol. 67, no. 1, pp. 1–31, 2020.
- [16] O. Schwartz and N. Vaknin, "Pebbling game and alternative basis for high performance matrix multiplication," *SIAM Journal on Scientific Computing*, vol. 45, no. 6, pp. C277–C303, 2023.
- [17] J. Shen, Y. Qiao, Y. Huang, M. Wen, and C. Zhang, "Towards a multi-array architecture for accelerating large-scale matrix multiplication on fpgas," in *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2018, pp. 1–5.
- [18] B. Barabasz and D. Gregg, "Winograd convolution for dnns: Beyond linear polynomials," in *AI* IA 2019—Advances in Artificial Intelligence: XVIIIth International Conference of the Italian Association for Artificial Intelligence, Rende, Italy, November 19–22, 2019, Proceedings*. Springer, 2019, pp. 307–320.
- [19] M. Vardhana and R. Pinto, "High-performance winograd based accelerator architecture for convolutional neural network," *IEEE Computer Architecture Letters*, 2025.
- [20] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [21] K. Chitsaz, M. Hajabdollahi, N. Karimi, S. Samavi, and S. Shirani, "Acceleration of convolutional neural network using fft-based split convolutions," *arXiv preprint arXiv:2003.12621*, 2020.
- [22] T. Highlander and A. Rodriguez, "Very efficient training of convolutional neural networks using fast fourier transform and overlap-and-add," *arXiv preprint arXiv:1601.06815*, 2016.
- [23] Y. Chen, L. De-Jian, F. Xi, C. Chong-Fei, W. Ji-Ceng, and L. Yu, "Fast fourier transform convolutional neural network accelerator based on overlap addition," *Journal of China Universities of Posts and Telecommunications*, vol. 31, no. 5, p. 71, 2024.
- [24] Y. Meng, J. Wu, S. Xiang, J. Wang, J. Hou, Z. Lin, and C. Yang, "A high-throughput and flexible cnn accelerator based on mixed-radix fft method," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.
- [25] H. Pratt, B. Williams, F. Coenen, and Y. Zheng, "Fcnn: Fourier convolutional neural networks," in *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2017, Skopje, Macedonia, September 18–22, 2017, Proceedings, Part I*. Springer, 2017, pp. 786–798.
- [26] S. Liu, H. Fan, and W. Luk, "Accelerating fully spectral cnns with adaptive activation functions on fpga," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1530–1535.
- [27] S. O. Ayat, M. Khalil-Hani, A. A.-H. Ab Rahman, and H. Abdellatif, "Spectral-based convolutional neural network without multiple spatial-frequency domain switchings," *Neurocomputing*, vol. 364, pp. 152–167, 2019.
- [28] Y. Han and B.-W. Hong, "Deep learning based on fourier convolutional neural network incorporating random kernels," *Electronics*, vol. 10, no. 16, p. 2004, 2021.
- [29] T. Watanabe and D. F. Wolf, "Image classification in frequency domain with 2srelu: a second harmonics superposition activation function," *Applied Soft Computing*, vol. 112, p. 107851, 2021.
- [30] N. Nguyen-Thanh, H. Le-Duc, D.-T. Ta, and V.-T. Nguyen, "Energy efficient techniques using fft for deep convolutional neural networks," in *2016 International Conference on Advanced Technologies for Communications (ATC)*. IEEE, 2016, pp. 231–236.
- [31] Q. Lan, Z. Wang, M. Wen, C. Zhang, and Y. Wang, "High performance implementation of 3d convolutional neural networks on a gpu," *Computational intelligence and neuroscience*, vol. 2017, no. 1, p. 8348671, 2017.
- [32] X. Li, Y. Chu, and X. Wu, "3d convolutional neural network based on spatial-spectral feature pictures learning for decoding motor imagery eeg signal," *Frontiers in Neuroinformatics*, vol. 18, p. 1485640, 2024.
- [33] X. Li, Y. He, W. Zhu, W. Qu, Y. Li, C. Li, and B. Zhu, "Split_composite: A radar target recognition method on fft convolution acceleration," *Sensors*, vol. 24, no. 14, p. 4476, 2024.
- [34] R. C. Lera and B. C. Albertini, "Hardware-efficient convolution algorithms for cnn accelerators: A brief review," in *Anais do XX Encontro Nacional de Inteligência Artificial e Computacional*. SBC, 2023, pp. 86–96.
- [35] E. Reis, T. Akilan, and M. Khalid, "Phasor-driven acceleration for fft-based cnns," *arXiv preprint arXiv:2406.00290*, 2024.
- [36] Y. Xu, J. Luo, and W. Sun, "Flare: An fpga-based full precision low power cnn accelerator with reconfigurable structure," *Sensors*, vol. 24, no. 7, p. 2239, 2024.
- [37] J. Jiang, Y. Zhou, Y. Gong, H. Yuan, and S. Liu, "Fpga-based acceleration for convolutional neural networks: A comprehensive review," *arXiv preprint arXiv:2505.13461*, 2025.
- [38] P. Antunes and A. Podobas, "Fpga-based neural network accelerators for space applications: A survey," *arXiv preprint arXiv:2504.16173*, 2025.
- [39] A. M. Salman, A. S. Tulan, R. Y. Mohamed, M. H. Zakhari, and H. Mostafa, "Comparative study of hardware accelerated convolution neural network on pynq board," in *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*. IEEE, 2020, pp. 578–582.
- [40] C. J. Faber, S. D. Harris, Z. Xiac, R. D. Chamberlain, and A. M. Cabrera, "Challenges designing for fpgas using high-level synthesis," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2022, pp. 1–7.
- [41] M. R. Ahmed, T. Koike-Akino, K. Parsons, and Y. Wang, "Autohls: Learning to accelerate design space exploration for hls designs," in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, 2023, pp. 491–495.
- [42] J. Lau, Y. Xiao, Y. Xie, Y. Chi, L. Song, S. Xiang, M. Lo, Z. Zhang, J. Cong, and L. Guo, "Rapidstream ir: Infrastructure for fpga high-level physical synthesis," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–11.

- [43] C. F. Gauss, "Nachlass: Theoria interpolationis methodo nova tractata," *Carl Friedrich Gauss Werke*, vol. 3, pp. 265–327, 1866.
- [44] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [45] S. W. Smith, *Convolution*. San Diego, CA, USA: California Technical Publishing, 1997, ch. 6, pp. 107–122.
- [46] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3d convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 4489–4497.
- [47] J. Volder, "The cordic computing technique," in *Papers presented at the the March 3-5, 1959, western joint computer conference*, 1959, pp. 257–261.
- [48] A. Banerjee, A. S. Dhar, and S. Banerjee, "Fpga realization of a cordic based fft processor for biomedical signal processing," *Microprocessors and Microsystems*, vol. 25, no. 3, pp. 131–142, 2001.
- [49] P. Choudhary and A. Karmakar, "Cordic based implementation of fast fourier transform," in *2011 2nd International Conference on Computer and Communication Technology (ICCT-2011)*. IEEE, 2011, pp. 550–555.
- [50] A. A. Karatsuba and Y. P. Ofman, "Multiplication of many-digital numbers by automatic computers," in *Doklady Akademii Nauk*, vol. 145, no. 2. Russian Academy of Sciences, 1962, pp. 293–294.
- [51] Z. Liu, P. Chow, J. Xu, J. Jiang, Y. Dou, and J. Zhou, "A uniform architecture design for accelerating 2d and 3d cnns on fpgas," *Electronics*, vol. 8, no. 1, p. 65, 2019.
- [52] Y. Yang, W. He, J. Hu, S. Cheng, W. Xu *et al.*, "Fpga-based design for accelerating 3d convolutional neural network," *International Journal of Frontiers in Engineering Technology*, vol. 5, no. 3, pp. 40–48, 2023.
- [53] X. Li, H. Huang, T. Chen, H. Gao, X. Hu, and X. Xiong, "A hardware-efficient computing engine for fpga-based deep convolutional neural network accelerator," *Microelectronics Journal*, vol. 128, p. 105547, 2022.
- [54] A. Kyriakos, E.-A. Papatheofanous, C. Bezaitis, and D. Reisis, "Resources and power efficient fpga accelerators for real-time image classification," *Journal of Imaging*, vol. 8, no. 4, p. 114, 2022.
- [55] M. Meyer, T. Kenter, and C. Plessl, "Evaluating fpga accelerator performance with a parameterized opencl adaptation of the hpc-challenge benchmark suite," *arXiv preprint arXiv:2004.11059*, 2020.
- [56] A. Montgomerie-Corcoran, Z. Yu, J. Cheng, and C.-S. Bouganis, "Pass: Exploiting post-activation sparsity in streaming architectures for cnn acceleration," in *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2023, pp. 288–293.
- [57] M. Courbariaux, Y. Bengio, and J.-P. David, "Training deep neural networks with low precision multiplications," *arXiv preprint arXiv:1412.7024*, 2014.
- [58] J. Kim, W. Jung, H. Kim, and J. Lee, "Cycnn: A rotation invariant cnn using polar mapping and cylindrical convolution layers," *arXiv preprint arXiv:2007.10588*, 2020.
- [59] H. Mo and G. Zhao, "Ric-cnn: rotation-invariant coordinate convolutional neural network," *Pattern Recognition*, vol. 146, p. 109994, 2024.
- [60] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, "Error analysis and improving the accuracy of winograd convolution for deep neural networks," *ACM Transactions on Mathematical Software (TOMS)*, vol. 46, no. 4, pp. 1–33, 2020.
- [61] S. A. Alam, A. Anderson, B. Barabasz, and D. Gregg, "Winograd convolution for deep neural networks: Efficient point selection," *ACM Transactions on Embedded Computing Systems*, vol. 21, no. 6, pp. 1–28, 2022.
- [62] S. Kala, B. R. Jose, J. Mathew, and S. Nalesh, "High-performance cnn accelerator on fpga using unified winograd-gemm architecture," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 12, pp. 2816–2828, 2019.
- [63] W. Shen, J. Jiang, M. Li, and S. Liu, "Efficient cordic-based activation functions for rnn acceleration on fpgas," *IEEE Transactions on Artificial Intelligence*, 2024.
- [64] O. Kokane, G. Raut, S. Ullah, M. Lokhande, A. Teman, A. Kumar, and S. K. Vishvakarma, "Retrospective: A cordic based configurable activation function for nn applications," *arXiv preprint arXiv:2503.14354*, 2025.
- [65] H. Sorensen, M. Heideman, and C. Burrus, "On computing the split-radix fft," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 34, no. 1, pp. 152–156, 2003.
- [66] C. Burrus and P. Eschenbacher, "An in-place, in-order prime factor fft algorithm," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 4, pp. 806–817, 1981.